

Prompt 1c — Disco, calendario de servicio y canario

Las cuatro correcciones de 1b quedaron bien aplicadas y verificadas. Estas cuatro salen de leer los resultados de esa verificación. La #1 es la urgente: llena el disco del VPS en tres semanas.

Repo: `github.com/leanchoi/leanchoi`, rama `claudes/esquel-data-ecosystem-integration-qbxuv2`.
`git pull --rebase` primero: hay cambios en `specs/sql/01_air_schema.sql`,
`specs/config/rutas_muestreo.json`, `docs/01-motor-aereos.md` y un archivo nuevo,
`specs/config/calendario_servicio.json`.

El trabajo de 1b está bien: crudo antes de parsear, reprocesamiento reproducible,
one-way bidireccional, hash determinista, tope sin truncar en silencio. Todo
verificado con pruebas reales, no declarado. Esto es afinado sobre eso.

=====

P0 – El crudo pesa 2 MB por consulta y va a llenar el disco

=====

Tu propia prueba lo midió: el archivo de la prueba de fallo forzado ocupa
2.085.158 bytes YA COMPRIMIDO. Eso es la página HTML entera de Google Flights.

172 consultas/día x 2,085 MB = 358 MB/día
x 90 días = 31 GB
con Flb completo (~300/día) = ~55 GB

Mi instrucción del turno pasado ("es JSON, pesa poco, subí la retención a 90
días") estaba mal por un factor de cincuenta. El problema aparece recién a las
tres semanas, cuando ya hay serie que perder, y un colector que llena el disco
no se rompe solo: se lleva puesto a Métrica y al tablero, que comparten VPS.

Corrección:

- Guardar el BLOB JSON que consume el parser, no la página entera. Es lo único
que hace falta para reprocesar y es uno o dos órdenes de magnitud más chico.
- La página completa queda solo como muestra de fixtures: 5 por día, 7 días.
- Presupuesto de disco declarado: 8 GB para bronce, con poda automática de lo
más antiguo y aviso en `meta.json` al superarlo.
- MEDÍ el tamaño real del blob en la primera corrida y reportalo. Si el blob
resulta ser >200 KB comprimido, avisá y recalculamos la retención.

Ver `global.retencion_crudo` en `rutas_muestreo.json`.

=====

P1 – El calendario de servicio hardcodeado deja pasar bloqueos como datos

=====

Dos problemas con la regla actual ("si toca EQS y es martes y devuelve 0 -> sin_servicio"):

a) NO CUBRE LAS RUTAS ESTACIONALES, que son las que más importan.

Tu corrida en vivo: COR→EQS el 2026-09-18 quedó en sin_resultados. Es viernes, y COR→EQS opera JUEVES de ida (agosto-septiembre, programa de Conectividad Sostenible). No hay servicio ese día: es un DATO registrado como HUECO.

b) UN BLOQUEO BLANDO EN UN DÍA SIN SERVICIO SE REGISTRA COMO DATO LEGÍTIMO.

Si Google devuelve HTTP 200 con una página vacía por soft-block y toca martes, la regla dice sin_servicio y NO descuenta cobertura. El sistema se auto-certifica sano justo cuando dejó de medir. Es el peor error posible del colector.

Corrección – sin_servicio exige LAS TRES condiciones, no dos:

1. cero itinerarios
2. respuesta_valida == true: la respuesta contiene evidencia de que el buscador ENTENDIÓ la consulta (metadatos de ruta/aeropuertos, no una página de error o interstitial). Esta es la que falta y la que discrimina de verdad.
3. el calendario versionado explica el cero.

Y la bitácora guarda los HECHOS, no la conclusión:

itineraries_por_aerolinea, respuesta_valida, calendario_explica, calendario_version.

Así la clasificación se puede RE-DERIVAR si el calendario cambia, sin volver a scrapear un pasado que ya no existe. La capa bronce registra hechos; las interpretaciones envejecen.

Usá specs/config/calendario_servicio.json (nuevo). Regla de default: para una ruta SIN entrada ahí, NUNCA se asume sin_servicio. Preferible subestimar la cobertura que inventar un dato.

Extra que sale gratis: acumulá el calendario DERIVADO de las observaciones y compará contra la semilla. Si difieren 3 semanas seguidas, gana el derivado. El servicio cambia por temporada y por acuerdo comercial, y ninguna fuente de terceros lo publica bien – fue justamente lo que nos hizo equivocarnos con las frecuencias de Esquel.

=====

P1 – El canario va a llorar lobo en la ruta núcleo

=====

BUE→BRC devuelve ~18 itinerarios por consulta. BUE→EQS devuelve 1 o 2. Con mediana 2, perder un itinerario es -50%: dispara el umbral de -30% por variación normal de disponibilidad, no por degradación del parser. En la ruta que más importa el canario alertaría todos los días, y un canario que se ignora es peor que no tener canario.

- Ruta densa (mediana de 7 días >= 5 itinerarios): caída de conteo, -30%.

- Ruta fina (mediana < 5): DESAPARICIÓN del operador donde antes aparecía, 3 corridas consecutivas.

En rutas finas la señal no es cuántos, es si el operador dejó de aparecer.

=====

P2 – Flb no entra en el tope: definí la política ahora

=====

Con el plan completo de Flb:

tiers 1-2 ancla (medido)	172	
tier 3 bidireccional /3	48	
tier 4 semanal	7	
rolling no-ancla /3	77	
checkpoints /7	9	
TOTAL	312	vs tope 250 -> se caen ~62/día

Que queden registradas como omitido_por_presupuesto ya está resuelto, pero eso las hace visibles, no correctas: hay que ELEGIR cuáles caen, no descubrirlo.

- Orden de prioridad explícito (ver global.politica_de_tope).
- Reducción previa: rolling de tier 3 cada 4 días, retorno de tier 3 solo en fechas ancla.
- Elevación escalonada del tope 250 -> 275 -> 300, y SOLO con evidencia: 14 días consecutivos con 0 bloqueos por paso, revirtiendo ante cualquier bloqueo en 7 días. Nunca subir de golpe: F0-3 validó 30 consultas, no 300 sostenidas.

=====

CRITERIOS DE ACEPTACIÓN

=====

1. Tamaño real del blob JSON medido y reportado; proyección a 90 días dentro del presupuesto de 8 GB.
2. La poda por presupuesto funciona: bajá el límite a 50 MB a propósito y verificá que poda lo más antiguo y avisa en meta.json.
3. COR→EQS un viernes de septiembre da sin_servicio, no sin_resultados.
4. Una respuesta vacía SIN respuesta_valida da sin_resultados aunque el calendario diga que no hay servicio. Probalo con un payload de interstitial.
5. Una ruta sin entrada en el calendario nunca produce sin_servicio.
6. El canario en modo ruta fina no dispara cuando BUE→EQS pasa de 2 a 1 itinerario, y sí dispara si AR desaparece 3 corridas seguidas.
7. El plan de Flb reporta su total y qué se cae por prioridad, sin sorpresas.

Y lo pendiente de Fla que sigue pendiente: las TRES NOCHES con cobertura >=90%, RSS y duración. Reportá el desglose por estado de cada noche: ok / sin_servicio / sin_resultados / bloqueado / parse_error / omitido_por_presupuesto.

